

ARKAND

Дизайн-система и архитектура

Финансовая CRM · Admin-панель · Единый визуальный стандарт

Проект	ARKAND — единая CRM-экосистема холдинга
Документ	Фирменные цвета · Архитектура backend (DRF) · Архитектура admin (React / FSD)
Стек	Django 5 + DRF · PostgreSQL · React 19 + TypeScript + Vite · FSD
Аудитория	Команда разработки (backend / frontend)
Версия	1.0
Дата	Июль 2026 г.
Исполнитель	WeBrand · ИП Бобоев Махмадсаид Мирзоалиевич

01 Технологический стек

общая договорённость

Единый стек для всех модулей экосистемы. Backend — модульный монолит на Django + DRF; frontend admin-панели — React по методологии Feature-Sliced Design. Общий язык интерфейса — русский и таджикский (i18n с первого дня).

Слой	Технологии	Назначение
Backend	Python 3.12 · Django 5 · Django REST Framework	API, бизнес-логика, права
База данных	PostgreSQL 16	транзакции, деньги, аудит
Фоновые задачи	Celery + Redis	расчёт зарплаты, отчёты, экспорт
Аутентификация	JWT (SimpleJWT) · роли и права	доступ, RBAC
Frontend	React 19 · TypeScript · Vite · FSD	admin-панель
Данные UI	TanStack Query · Axios · Zod	запросы, кэш, валидация
Локализация	i18next (ru / tj)	двухязычный интерфейс
Инфраструктура	Docker · Nginx · Hetzner / Coolify	production, всегда онлайн

Принцип: один backend, один frontend, единые контракты API и единая дизайн-система на все бизнесы холдинга. Новые модули добавляются как отдельные Django-приложения и FSD-слайсы, не ломая существующие.

02 Фирменные цвета

дизайн-токены

Палитра выведена из логотипа: вишнёвый феникс, глубокое бордо теней, тёплый чёрный вордмарка. Один набор токенов на весь интерфейс — цвета не подбираются «на глаз» в компонентах, а берутся из переменных.

Ядро бренда

#A4161A Arkand Crimson Основной · кнопки, акценты	#6B0F18 Deep Maroon Hover · глубина, тени	#1A1416 Warm Ink Заголовки, текст	#FAF9F8 Paper Фон страницы
--	--	--	---

Arkand Crimson — шкала (600 — основной)

50	100	200	300	400	500	600	700	800	900	950
FDF2F3	FADEE0	F3BCC0	E88F96	D85E68	C22530	A4161A	85111A	6B0F18	4E0B12	2E070A

Нейтральные — тёплый серый

0	50	100	200	300	400	500	600	700	800	900
---	----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Статусы интерфейса

Success Готово · подтверждено #15803D	Warning Лимит · ожидает #B45309	Error Ошибка · удаление #DC2626	Info Нейтральный статус #1D4ED8
---	---	---	---

Финансовые значения

Доход +284 500	Расход −96 200	Баланс 188 300
--------------------------	--------------------------	--------------------------

Бренд ≠ деньги. Расход и отрицательный баланс — отдельный сигнальный красный #DC2626, а не вишнёвый бренд-цвет. Вишнёвый работает только на кнопках и акцентах, чтобы «минус по кассе» визуально не сливался с айдентикой. Доход — зелёный, ошибки отличаются от «денежного минуса» иконкой и фоном, а не только цветом.

Категориальные — графики по бизнесам

 Крас Бирюза Янтарь Сланец Слива Зелень

CSS-переменные (копировать в проект)

```
/* shared/config/tokens.css */
:root{
  /* бренд */
  --brand:#A4161A;      --brand-hover:#85111A;  --brand-deep:#6B0F18;
  --ink:#1A1416;        --paper:#FAF9F8;        --white:#FFFFFF;
  /* нейтральные (warm gray) */
  --n-50:#FAF9F8;  --n-100:#F4F2F1;  --n-200:#E7E3E2;  --n-300:#D4CFCD;
  --n-400:#B0A9A7;  --n-500:#8A8280;  --n-600:#665F5E;  --n-700:#4A4341;
  --n-800:#2E2826;  --n-900:#1A1416;
  /* статусы */
  --success:#15803D;  --warning:#B45309;  --error:#DC2626;  --info:#1D4ED8;
  /* деньги */
  --money-in:#15803D;  --money-out:#DC2626;  --money-zero:#1A1416;
}
```

Три правила

- **Вишнёвый зарезервирован.** Только первичные кнопки, активная навигация, бренд-акценты. Не использовать для денег, статусов и фонов.
- **Деньги — отдельная семантика.** Приход/доход — зелёный, расход/минус — #DC2626, ноль/баланс — тёмный. Одинаково во всех модулях и графиках.
- **Нейтральные тёплые.** У серых лёгкий красный подтон — они дружат с вишнёвым. Не подмешивать холодные синевато-серые.

03 Backend — архитектура (DRF)

модульный монолит

Архитектура — **модульный монолит с сервисным слоем**. Каждый домен холдинга это отдельное Django-приложение; внутри приложения логика разложена по слоям. Финансы требуют строгости: деньги, транзакции и права проверяются в одном месте, а не размазаны по вьюхам.

Слои внутри модуля

Слой	Ответственность	Правило
<code>models.py</code>	структура данных, инварианты на уровне записи	без бизнес-логики
<code>selectors.py</code>	чтение: выборки, фильтры, агрегации	только read
<code>services.py</code>	запись: вся бизнес-логика, транзакции	только здесь пишем в БД
<code>serializers.py</code>	валидация входа, форма ответа	без логики предметной области
<code>views.py</code>	тонкие ViewSets: принять → вызвать сервис → отдать	никакой логики
<code>permissions.py</code>	RBAC + object-level доступ	роли и «своё/чужое»

Главное правило: бизнес-логика живёт в `services.py` и `selectors.py`. Views и serializers остаются тонкими. Это даёт тестируемость, переиспользование (сервис зовётся и из API, и из Celery, и из management-команды) и предсказуемость для команды.

Структура проекта

```
backend/
├── config/           # settings, urls, celery, asgi/wsgi
├── apps/
│   ├── core/        # Business, справочники, базовые модели (Часть 0)
│   ├── accounts/    # пользователи, роли, права
│   ├── finance/     # приходы/расходы, прибыль (ФНС-01...04)
│   ├── cash/        # кассы, лимиты оборота (КАС-01...04)
│   ├── settlements/ # взаиморасчёты, долги, бартер (БАР-01...04)
│   ├── payroll/     # зарплата, гибкие схемы бонусов (ЗРП-01...05)
│   ├── reports/     # сводные отчёты (ФНС-10...13)
│   └── audit/       # журнал всех действий
└── manage.py
```

Разложение модуля (пример finance/)

```
finance/
├─ models.py      selectors.py    services.py
├─ serializers.py views.py      permissions.py
├─ filters.py     urls.py       tests/
```

Финансовые инварианты — обязательны

- **Только Decimal для денег** — `DecimalField(max_digits=14, decimal_places=2)`.
Никогда `float`.
- **Атомарность** — любая денежная операция в `transaction.atomic()`; при гонках — `select_for_update()`.
- **Идемпотентность** — повтор запроса не создаёт вторую операцию (ключ идемпотентности / проверка статуса).
- **Аудит** — каждое изменение денег и статусов пишется в `audit` (кто, что, когда, до/после).
- **Soft-delete** — финансовые записи не удаляются физически, а помечаются; для отчётности и истории.
- **Изоляция касс** — «касса видит только своё» реализуется фильтрацией `queryset + object-level permission`, а не скрыванием в UI.

Пример: модель и сервис

```
# finance/models.py
class Transaction(TimeStampedModel):
    business = models.ForeignKey("core.Business", on_delete=models.PROTECT)
    kind = models.CharField(choices=TxKind.choices) # income | expense
    category = models.ForeignKey("finance.ExpenseCategory", null=True,
on_delete=models.PROTECT)
    amount = models.DecimalField(max_digits=14, decimal_places=2) # деньги = Decimal
    method = models.CharField(choices=PayMethod.choices) # cash | transfer
    status = models.CharField(default=TxStatus.PENDING)
    confirmed_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
on_delete=models.SET_NULL)
```

```
# settlements/services.py — авто-фиксация долга при одобренной передаче (ХОЛ-30)
@transaction.atomic
def approve_transfer(*, transfer: Transfer, actor: User) -> Debt:
    transfer.mark_approved(actor) # смена статуса
    debt = Debt.objects.create(
        debtor=transfer.to_business, creditor=transfer.from_business,
        amount=transfer.amount, source=transfer, # прозрачный реестр
    )
    AuditLog.record(actor, "transfer.approved", transfer)
    return debt
```

Конвенции API

Тема	Правило
Версионирование	<code>/api/v1/...</code>

Аутентификация	JWT (access + refresh), права на каждый endpoint
Списки	пагинация + фильтры (django-filter) + сортировка
Ошибки	единый формат { code, message, details }
Деньги в JSON	строка, не число с плавающей точкой

04 Admin — архитектура (React / FSD)

Feature-Sliced Design

Admin-панель строится по **Feature-Sliced Design**: код разложен по слоям от общего к частному, импорты идут строго сверху вниз. Это масштабируется под холдинг — новый модуль добавляется как набор слайсов, не задевая остальные.

Слои FSD

Слой	Что внутри	Пример для ARKAND
app	провайдеры, роутер, тема, i18n, глобальные стили	токены, ProtectedRoute, layout
pages	страницы-композиции из виджетов	Финансы, Кассы, Долги, Зарплата, Отчёты
widgets	самостоятельные блоки страницы	сайдбар, таблица операций, карточка кассы
features	действия пользователя (сценарии)	добавить приход, подтвердить операцию, закрыть долг
entities	бизнес-сущности и их представление	transaction, cash-register, debt, employee, business
shared	ui-kit, api-client, lib, config — без бизнес-смысла	кнопки, поля, токены, axios, форматтеры денег

Правило импортов: слой видит только слои **ниже** себя. pages → widgets → features → entities → shared. Слайсы на одном слое между собой не импортируются напрямую. Это и есть защита от «спагетти».

Структура проекта

```
src/
├─ app/           # провайдеры, роутер, тема, i18n
├─ pages/         # finance, cash, settlements, payroll, reports
├─ widgets/       # sidebar, tx-table, cash-card, debt-registry
├─ features/      # add-income, confirm-tx, close-debt, payroll-run
├─ entities/      # transaction, cash-register, debt, employee, business
└─ shared/        # ui, api, lib, config (tokens.css)
```

Структура слайса

```
entities/transaction/
├─ ui/            # TxRow, TxBadge, TxAmount (цвет по знаку суммы)
├─ model/         # типы, стор, селекторы
├─ api/           # запросы к /api/v1/transactions
└─ index.ts       # публичный контракт слайса
```

Стек по слоям

Инструмент	Роль
React 19 + TypeScript + Vite	каркас и сборка
TanStack Query + Axios	запросы, кэш, инвалидация (в entities/*/api)
Zod	валидация ответов API и форм
i18next	ru / tj с первого дня
CSS-переменные (tokens.css)	дизайн-система из раздела 02 — единый источник цвета

Связка с дизайн-системой: все цвета в компонентах — только через переменные --brand, --money-in/out, --n-* из shared/config/tokens.css. Хардкод hex в компонентах запрещён — так весь холдинг остаётся в одной палитре.

05 Параллельная разработка

как делить работу

Модульность backend и FSD frontend позволяют команде работать параллельно с минимумом конфликтов. Точка стыковки — контракт API: как только он согласован, backend и frontend движутся независимо.

Backend — по доменам

- **Разработчик А** — core + accounts + права (фундамент для всех).
- **Разработчик В** — finance + cash (приходы/расходы, кассы, лимиты).
- **Разработчик С** — settlements + payroll + reports (долги, зарплата, отчёты).

Frontend — по фичам

- Общий shared/ui + токены делает один человек первым.
- Дальше страницы берутся параллельно: Финансы, Кассы, Долги, Зарплата, Отчёты.
- Сущности (entities/*) переиспользуются между страницами.

Что согласовать до старта кода: Часть 0 (список бизнесов, пользователи, роли), правила ХОЛ-30...33 (логика долгов и взаиморасчётов) и формат данных для надстройки владельцев (Часть 7). Дизайн-система и стек в этом документе уже зафиксированы.